Practical 8

Programs on Libraries, Assemblies, Packages, Nuget and Parallel Programing

Library
A Class Library in C# is a project that contains reusable classes and methods. It is compiled into a DLL and can be referenced by other C# applications.

Assembly
An **assembly** is a compiled unit of a .NET application. It can be an .exe or .dll file and contains compiled code, metadata, and other information.

Package
A **package** is a collection of reusable code distributed so that developers can add functionality to their applications without creating everything from scratch.

Parallel programming
Parallel programming means executing multiple operations at the same time using multiple processor cores. C# provides the **Task Parallel Library (TPL)** through classes such as Parallel, Task, and Parallel.For.

8.1. Using a .NET Class Library

```
using System;
namespace LibraryDemo
{
   class Program
   {
      static void Main(string[] args)
      {
         double number = 25;
         double squareRoot = Math.Sqrt(number);
         double power = Math.Pow(5, 2);
         Console.WriteLine("Number: " + number);
         Console.WriteLine("Square Root: " + squareRoot);
         Console.WriteLine("5 raised to power 2: " + power);
         Console.ReadLine();
      }
   }
}
```

Output

```
Number: 25
Square Root: 5
5 raised to power 2: 25
```

8.2. Creating a Class Library

Open **Visual Studio 2019**.

1: Start a new project

1. Open **Visual Studio 2019**.
2. Click **Create a new project**.
3. Select **Class Library (.NET Framework)** if you are using the .NET Framework.
4. Click **Next**.

2. Configure the Project

Enter the following details:

Field	Value
Project Name	MyCalculatorLibrary
Location	Choose any folder
Solution Name	ClassLibraryDemo
Framework	.NET Framework 4.7.2 or the version available

Click **Create**.
Visual Studio creates a project containing a file called:
Class1.cs

3. Create a Calculator Class

Right-click **Class1.cs** in Solution Explorer and select:

Rename → Calculator.cs

Replace the existing code with:

```
using System;
namespace MyCalculatorLibrary
{
   public class Calculator
   {
      public int Add(int a, int b)
      {
         return a + b;
      }

      public int Subtract(int a, int b)
      {
         return a - b;
      }

      public int Multiply(int a, int b)
      {
```

```
            return a * b;
        }

        public double Divide(int a, int b)
        {
            if (b == 0)
                throw new DivideByZeroException();

            return (double)a / b;
        }
    }
}
```

4. Build the Class Library

From the Visual Studio menu, select:

Build → Build Solution

If there are no errors, Visual Studio creates a DLL file.

For example:

MyCalculatorLibrary.dll

The DLL is normally located under a folder such as:

bin\Debug\

A Class Library is compiled into a **DLL (Dynamic Link Library)** that can be reused by other applications.

5. Create a Console Application

Now create another project to use the Class Library.

Right-click the **Solution** in Solution Explorer. .(In **MyCalculatorLibrary** project only)

Add → New Project

Choose:

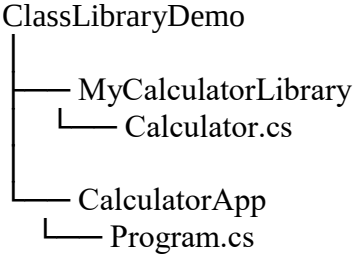
Console App (.NET Framework)

Name it:

CalculatorApp

Click **Create**.

Your solution now contains two projects:



6. Add a Reference to the Class Library

The Console Application needs to know about the Class Library.

Step 1

Right-click:

CalculatorApp

in Solution Explorer.

Select:

Add → Reference...

Step 2

Select:

Projects → Solution

You should see:

MyCalculatorLibrary

Select the checkbox beside it.

Click:

OK

Now the Console Application can use the classes contained in the Class Library.

7. Use the Class Library

Open:

Program.cs
```
using System;
```

```
using MyCalculatorLibrary;

namespace CalculatorApp
{
   class Program
   {
      static void Main(string[] args)
      {
         Calculator calc = new Calculator();

         Console.WriteLine("Addition: " + calc.Add(10, 5));
         Console.WriteLine("Subtraction: " + calc.Subtract(10, 5));
         Console.WriteLine("Multiplication: " + calc.Multiply(10, 5));
         Console.WriteLine("Division: " + calc.Divide(10, 5));

         Console.ReadLine();
      }
   }
}
```

Note: In Solution Explorer right click on CalculatorApp project and select **Set as Startup Project.**

8. Run the Program

The output will be:

```
Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2
```

Practical example

A college management system could have separate libraries:

```
CollegeManagement
│
├── StudentLibrary
│   └── Student.cs
│
├── FeesLibrary
│   └── Fees.cs
│
├── ExamLibrary
│   └── Exam.cs
│
└── MainApplication
    └── Program.cs
```

8.3. Display Assembly Information

```
using System;
using System.Reflection;

namespace AssemblyDemo
{
   class Program
   {
      static void Main(string[] args)
      {
         Assembly assembly = Assembly.GetExecutingAssembly();

         Console.WriteLine("Assembly Name: " +
            assembly.GetName().Name);

         Console.WriteLine("Assembly Version: " +
            assembly.GetName().Version);

         Console.WriteLine("Assembly Location: " +
            assembly.Location);

         Console.ReadLine();
      }
   }
}
```

Sample Output

```
Assembly Name: AssemblyDemo
Assembly Version: 1.0.0.0
Assembly Location: C:\...\AssemblyDemo.exe
```

8.4. Using a NuGet Package

Step 1: Create Console Application

Create:

NuGetDemo

in Visual Studio 2019.

Step 2:

Install the package.

Tools → NuGet Package Manager → Package Manager Console

and execute:

Install-Package Newtonsoft.Json

Step 3: Program

```
using System;
using Newtonsoft.Json;

namespace NuGetDemo
{
   class Student
   {
      public int Id { get; set; }
      public string Name { get; set; }
      public double Marks { get; set; }
   }

   class Program
   {
      static void Main(string[] args)
      {
         Student student = new Student
         {
            Id = 101,
            Name = "Rahul",
            Marks = 85.5
         };
         string json = JsonConvert.SerializeObject(student);
         Console.WriteLine("JSON Data:");
         Console.WriteLine(json);
         Console.ReadLine();
      }
   }
}
```

Output

```
JSON Data:
{"Id":101,"Name":"Rahul","Marks":85.5}
```

Here:

`using Newtonsoft.Json;`
allows the program to use functionality provided by the NuGet package.

8.5. Student Marks Processing Using Parallel Programming

Marks of 10 students and want to calculate their grades in parallel.

```
using System;
using System.Threading.Tasks;
namespace StudentParallelDemo
```

```
{
   class Program
   {
      static void Main(string[] args)
      {
         int[] marks = { 85, 72, 91, 65, 48, 78, 89, 55, 94, 69 };
         Parallel.For(0, marks.Length, i =>
         {
            string grade;

            if (marks[i] >= 80)
               grade = "A";
            else if (marks[i] >= 60)
               grade = "B";
            else if (marks[i] >= 40)
               grade = "C";
            else
               grade = "F";
            Console.WriteLine(
               "Student " + (i + 1) +
               " Marks: " + marks[i] +
               " Grade: " + grade);
         });
         Console.ReadLine();
      }
   }
}
```

Sample Output

```
Student 1 Marks: 85 Grade: A
Student 2 Marks: 72 Grade: B
Student 3 Marks: 91 Grade: A
Student 4 Marks: 65 Grade: B
Student 5 Marks: 48 Grade: C
...
```